



Augmentation des données pour l'imagerie médicale

Projet Personnel



20 JUIN 2026

RAJAA LEBNAITI

Table des matières

1) Plan de travail :	3
1-1) Préparation et pipeline des données	3
2-1) L'architecture et les fonctions de pertes	3
3-1) Entraînement et optimisation	3
4-1) Validation et déploiement	3
2) Notes	4
• L'asymétrie des classes	4
• La sensibilité aux détails	4
• L'architecture du WGAN-GP	4
• Validation des résultats	4
• L'interface Streamlit	5

Augmentation des données pour l'imagerie médicale

1) Plan de travail :

1-1) Préparation et pipeline des données

- Dataset de « Brain Tumor Detection » : des IRM du cerveau avec ou sans tumeur.
- Le script de pré-traitement : redimensionnement en 64*64, normalisation entre -1 et 1.
- Création du DataLoader avec Pytorch, et visualisation avec *matplotlib*.

2-1) L'architecture et les fonctions de pertes

- Conditional WGAN-GP
- Les classes *Generator* et *Critic* du GAN
- Les fonctions *get_gradient()*, *gradient_penalty()*, *get_critic_loss()*

3-1) Entraînement et optimisation

- Entraînement sur google colab ou kaggle pour la carte graphique
- Enregistrer une grille de 5 images toutes les 5 époques pour suivre visuellement le progrès du générateur.
- Calcul du score FID en fin d'entraînement

4-1) Validation et déploiement

- Faire le test du classificateur pour valider le GAN.
- Interface Streamlit
- Pousser le code sur GITHUB avec un fichier README comportant l'animation de l'évolution des images au fil des époques
- Déploiement sur Huggin Face Spaces.

2) Notes

+ L'asymétrie des classes

90 % d'images de patients sains et seulement 10 % d'images de patients malades.
Ton GAN va servir à équilibrer cela.

+ La sensibilité aux détails

Contrairement à un visage où un pixel flou passe inaperçu, sur une radiographie, un pixel flou ou un artefact bizarre peut être confondu avec une tumeur, donc le modèle doit être ultra-précis.

+ L'architecture du WGAN-GP

- Le Générateur :

Il prendra en entrée un bruit z (taille 100) + le label (0 pour sain, 1 pour malade) sous forme de vecteur One-Hot.

On utilise des blocs ConvTranspose2d avec la configuration ($K=4$, $S=2$, $P=1$) pour faire grandir l'image proprement jusqu'à $64 * 64$ ou $128 * 128$.

- Le Critique :

Il prendra l'image médicale + le label étiré sous forme de matrice 2D.

Sortie linéaire brute (pas de Sigmoid). On utilise `nn.InstanceNorm2d` ou `nn.LayerNorm2d` à la place de la `BatchNorm2d` car on applique le Gradient Penalty.

+ Validation des résultats

- La métrique (FID) Fréchet Inception Distance :

On compare les distributions statistiques des vraies images avec les fausses images.
Plus le score FID est proche de 0 plus les images générées sont réalistes.

`torchmetrics.image.fid.FrechetInceptionDistance`

- Test du classificateur :

On entraîne un petit réseau de classification classique (ex : un ResNet simple) uniquement sur les vraies images. On note sa précision (ex : 80 %).

On ajoute les images générées par le GAN pour gonfler le dataset d'entraînement.

On ré-entraîne le même classificateur. Si sa précision passe à 85 % ou 88 % cela prouve scientifiquement que le GAN génère des images contenant des informations médicales utiles et pertinentes

L'interface Streamlit

L'utilisateur arrive sur une page web et choisit une option via un bouton radio : [Sain] ou [Tumeur]

Il clique sur le bouton "Générer une image médicale".

L'application charge les poids de modèle entraîné (.pth), passe le bon label dans le Générateur, applique la dénormalisation, et affiche la radiographie artificielle instantanément à l'écran.